实验环境搭建及使用

杜忱莹 周辰霏
2021 年 2 月
周浩 李世阳 麦隽韵
2022 年 2 月
贺祎昕 徐一帆
2023 年 2 月
陈静怡 李君龙
2023 年 3 月
丁延峰 曹珉浩
2024 年 2 月
孔德嵘 马浩祎
2025 年 3 月
莫骐骥 王卓然
2026 年 3 月

目录

1 整体概述 **3**

2 SIMD 和多线程实验环境 **3**

2.1 x86 平台 4

2.1.1 TDM-GCC 编译器 4

2.1.2 集成开发环境 Code::Blocks 4

2.1.3 开发环境 Visual Studio Code 6

2.2 Linux 系统和 ARM 平台 7

2.2.1 WSL 8

2.2.2 Linux 虚拟机 9

2.2.3 华为鲲鹏服务器 9

2.2.4 VSCode 连接远程服务器 10

3 MPI 实验环境 **10**

3.1 自行搭建 10

3.1.1 MS-MPI 10

3.1.2 OpenMPI 10

3.1.3 MPICH 10

3.2 鲲鹏服务器 10

3.2.1 pbs 脚本配置 10

3.2.2 作业提交 11

4 GPU 实验环境 **12**

4.1 英伟达 CUDA 12

4.1.1 个人电脑环境搭建 12

4.2 AMD ROCm 12

4.2.1 AMD 云端学习平台 12

5 性能测试方法 **14**

5.1 问题规模 15

5.2 测试数据 15

5.3 样例 15

5.4 Windows 下精确计时 17

1 整体概述

同学们好, 并行课程所要配置的环境比较复杂, 实验方面总共有 6 次, 分别是**体系结构编程, SIMD 编程，多线程编程，多进程编程，GPU 编程，期末综合实验**。需要大家注意的是，体系结构编程作为大家初步上手的热身实验，是不需要在服务器进行测试的，大家本地测试性能即可。对于后面的 4 次小实验的环境，我们会在后面的章节依次进行介绍，大家可以一次性配置完毕，也可以在每次实验前配好相关环境（至少这几天不会很心累）。

2 SIMD 和多线程实验环境

本学期课程前期，同学们需要进行 SIMD 和多线程编程实验。SIMD（Single Instruction Multiple Data），即单指令多数据运算，是一种常见的并行架构，可方便实现数据并行、提高运算效率。SIMD 硬件结构有三种变体：向量体系结构、多媒体 SIMD 指令集扩展和 GPU。多线程是指从软件或者硬件上实现多个线程并发执行的技术，是为了提高计算资源利用率、提高程序运行效率。支持多线程的硬件包括对称多处理机（SMP）、多核 CPU 等。当前的计算机系统，从超级计算机到桌面 PC，甚至移动终端，普遍提供了对 SIMD 和多线程的支持。例如，x86 CPU 的 SSE/AVX 指令集、ARM CPU 的 Neon/SVE 指令集实现了 SIMD 运算；两种 CPU 目前也都采用多核架构、可很好支持多线程运算。考虑到 ARM 逐渐进入云计算领域的趋势，本学期的课程预计要求同学们在 SIMD 实验开始后使用 ARM 平台进行实验（具体见作业要求）。

除了硬件支持之外，SIMD 和多线程并行还需要操作系统和编程工具的支持。Linux 是一套免费使用和自由传播的类 Unix 操作系统，是云计算、高性能计算等领域占统治地位的操作系统。在我们未来的科研和工作环境，几乎必然会接触 Linux 系统的使用和开发。因此，希望同学们能基于 Linux 系统进行本课程的实验。Linux 传统的开发方式是通过命令行进行操控（虽然现在可视化开发环境也很成熟了），如果同学们以前没有过相关的经验，希望能够认真查阅一下相关资料，难度曲线可能在一开始会稍陡峭一些，但熟悉之后往往能够比鼠标操控的效率高得多。

GCC 全称为 GNU Compiler Collection，将是我们使用的主要编程工具。GCC 提供对 Pthread 多线程编程和 SIMD 编程的良好支持，因此我们 SIMD 和多线程的实验将主要基于 GCC（其实 MPI 和 GPU 的实验也是）。同时，GCC 也是工业级别工具，也希望借此课程增加同学们对它的了解。下面给出 GCC 编译课程所需要并行程序的编译选项示例：

```
gcc -march=corei7-avx     # SSE/AVX
gcc -mcpu=cortex-a76 -mfpu=neon-vfpv4 # NEON
gcc -pthread              # pthread
gcc -fopenmp              # OpenMP
```

git 较大规模的项目往往需要进行版本控制，比如进行代码回溯、分支管理，git 便是最常使用的工具。你可以在这里对它了解更多。同时为了避免实验环境的损坏等原因导致的意外，希望同学们将代码托管至云端 (如github)。此外，commit 记录也是独立完成作业的一种证明。

make 一个大型项目往往有着复杂的文件依赖关系，编译也是一个耗时的工作。make 便是用于辅助管理复杂的编译的依赖，并指定编译指令的程序。当然，为了程序员的方便，它引入了更多特性如伪文件，方便项目的管理与测试。你可以在这个或这个网站上了解更多。

对于课程的体系结构编程和后续的自行测试环节，可选择的几种开发环境部署方式总结如下：

- x86 平台、Windows 系统、TDM-GCC 编译器 +Code::Blocks 或 Visual Studio Code，这是最简单的一种组合，大家只需在熟悉的 Windows 系统中安装 TDM-GCC 编译器，使用 Code::Blocks 或 Visual Studio Code 进行开发即可。
- x86 平台、Windows 系统 +WSL、GCC 编译器，在 Windows 系统中直接进行 Linux 开发。
- x86 平台，Windows 系统 +Ubuntu 虚拟机、GCC 编译器，需安装 Linux 虚拟机
- x86 平台、Windows 系统 +WSL+QEMU、GCC 编译器，在 Windows 系统中模拟 ARM 平台 Linux 系统开发，注意，只用于正确性调试，**性能测试结果并非真实 ARM 平台性能表现**。
- ARM 平台（华为鲲鹏服务器）、GCC 编译器，真正的 ARM 平台 Linux 系统。

对于体系结构编程后的编程任务，要求大家使用华为鲲鹏服务器进行测试，具体测试方法会在 SIMD 实验前发出，以下的华为鲲鹏服务器的测试仅供参考。

接下来几节将介绍这些组合的安装和简单使用。

2.1 x86 平台

x86 架构（The x86 architecture）最早是 Intel 提出的微型处理器架构和指令集，经过三十余年的发展，在目前的个人 PC、服务器乃至超算中，x86 架构占据了很大比例。自 1996 年的 MMX 指令集开始，x86 体系不断完善其 SIMD 指令集，而从 2004 年左右 x86 架构开始支持多核，所以 x86 的硬件平台完全支持 SIMD 和多线程。对于 Windows 平台，推荐同学们使用 TDM-GCC 编译器和集成开发环境 Code::Blocks 进行实验。

2.1.1 TDM-GCC 编译器

Windows 本身没有内置 GCC 编译器，因此我们需要自行安装，下载地址：https://jmeubank.github.io/tdm-gcc/download/。TDM-GCC 是 Windows 平台下的一个免费的 GCC 编译套件，包含截止到 10.3.0 版的 GCC 稳定版本 +Windows 平台必要的补丁和运行时 API，支持 32 位和 64 位系统，非常小巧的单文件安装包（57.6MB），安装非常简单，且直接支持 SIMD、多线程等编程，可作为相关实验的编译器。

如果你需要更高版本的 GCC 编译器（本课程暂不需要），请考虑下载 MinGW-w64，MinGW-w64 直接同步 GCC 官方版本，能用到最新的语言特性（比如 C++23），但配置会更复杂一点。Github 下载地址：https://github.com/niXman/mingw-builds-binaries/releases。TDM-GCC 和 MinGW-w64 的编译命令（gcc、g++）完全一致。

2.1.2 集成开发环境 Code::Blocks

下载地址：http://www.codeblocks.org/downloads/。

Code::Blocks 是免费开源的跨平台集成开发环境，支持多种编译器：GCC、MSVC++、clang 等。具有完善的调试功能，可配合 TDM-GCC 形成完整开发环境。

- **编译器目录设置**

安装好的 Code::Blocks 需要设置编译器目录。如图2.1所示，选择 “Settings-Complier” 菜单项，在 “Toolchain executables” 表单中填入 TDM-GCC 安装地址和编译程序名即可。

- **创建项目**

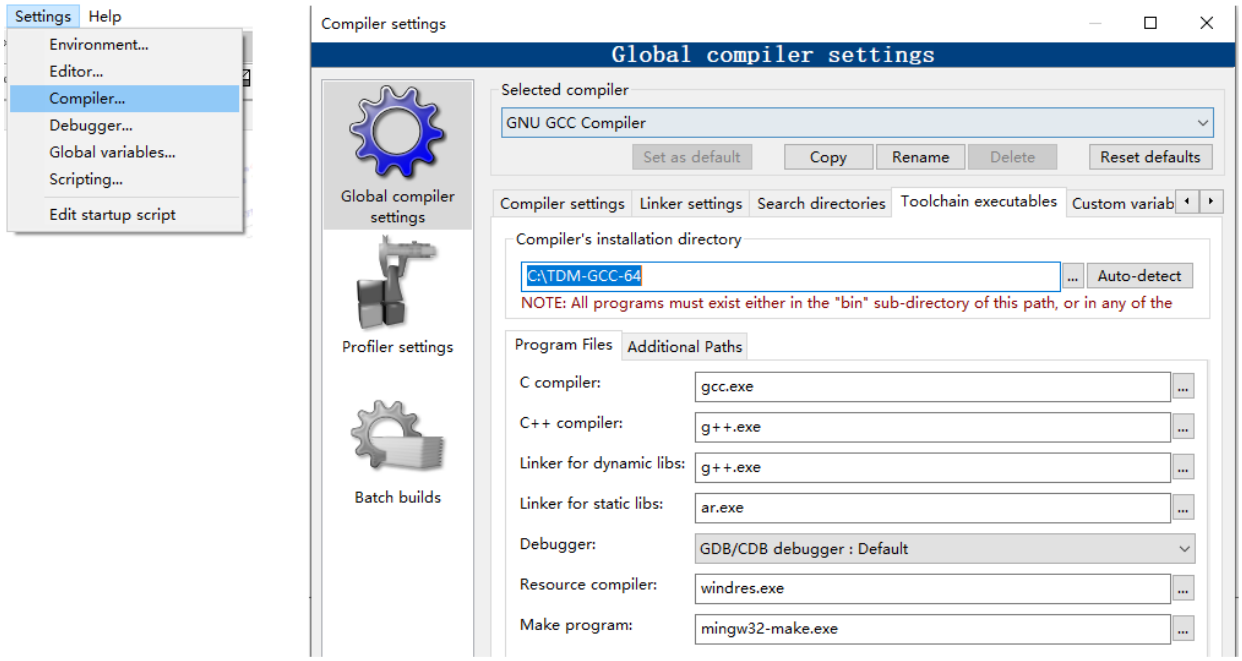

图 2.1: 设置编译器目录

创建项目过程如图2.2所示，选择 “File-New-Project” 菜单项，选择创建 “Console Application” 即可（我们的实验基本都是命令行程序），后续过程与 Visual Studio 中创建项目非常相似。

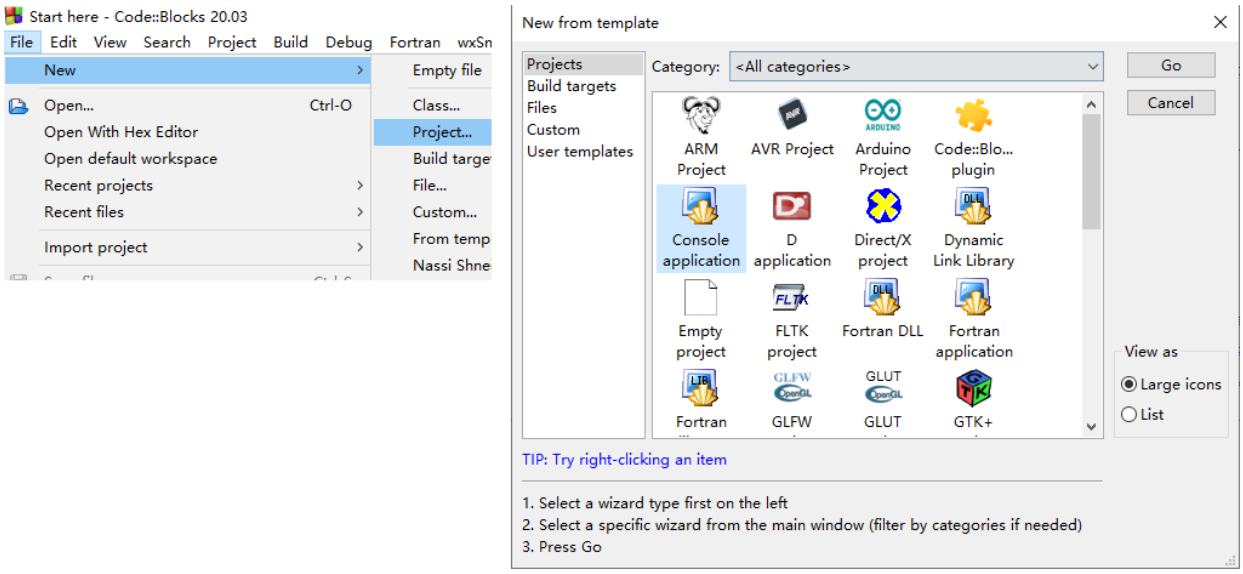

图 2.2: 创建项目

- **编译选项设置**

编译选项设置过程如图2.3所示，选择 “Project-Build options” 菜单项，在打开的对话框中的 “Complier Flags” 表单内勾选（GCC）编译选项。例如，选择 C++ 标准（TDM-GCC 9.2 目前只支持到 C++17）、选择目标程序 32 位或 64 位、选择优化力度、选择体系结构（编译 SSE/AVX 程序需要用到，编译 AVX 程序还需在 “Other Compiler options” 表单的文本框中填写 “-march=native”）等。编译 OpenMP 程序还需在 “Other compiler options” 表单的文本框中填写 “-fopenmp”、并在在 “Linker

settings” 表单的 “Other linker options” 文本框中填写 “-fopenmp” 等。

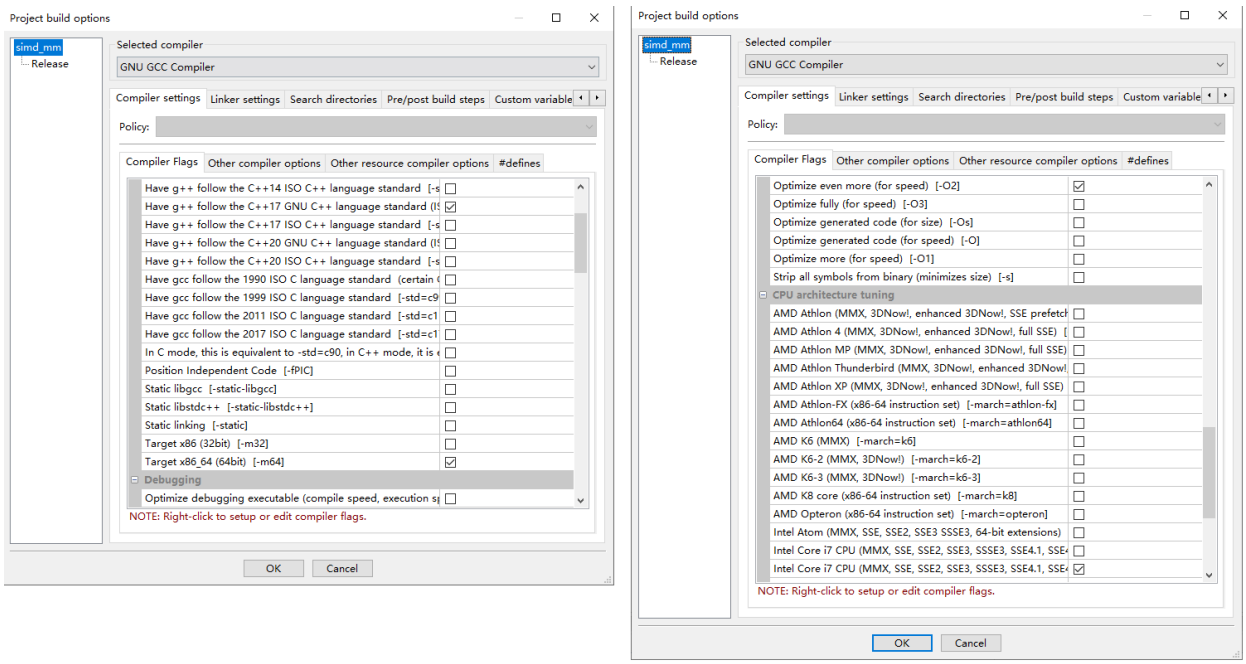

图 2.3: 编译选项

- **编译/运行项目**

接下来即可选择 “Build” 菜单编译和运行项目，以及选择 “Debug” 菜单调试项目，与 Visual Studio 的过程也类似。程序执行结果如图2.4所示。

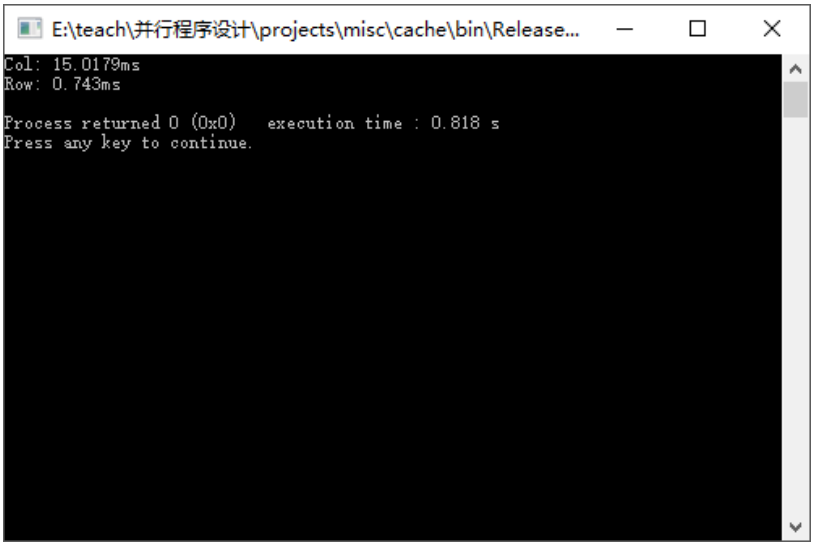

图 2.4: Code::Blocks 运行程序效果

2.1.3 开发环境 Visual Studio Code

Visual Studio Code（简称 VSCode，注意不是 Visual Studio）是微软推出的一款轻量级、免费但功能强大的代码编辑器，其特点是启动快、占用小、支持多种编程语言、插件丰富，只需安装微软官

方 C/C++ 插件即可实现语法高亮、代码跳转和补全；内置对 Git 的完善支持，可直接进行代码提交、同步与上传到 GitHub。它本质上不是一款 IDE，同学们可以在 VSCode 的内置终端采用纯命令行模式（有助于适应 Linux 环境下的编译与开发）进行编译、执行代码，也可以自行配置 json 文件实现一键编译与调试。在本科课程的后续学习中，掌握 VSCode 下的开发方式是非常有必要的，它能有效提升代码编写效率，适配更多课程与科研实践场景。下载地址：https://code.visualstudio.com。

- **创建文件**

点击左侧资源管理器，点击新建文件夹，然后点击鼠标右键进行新建文件即可。创建文件的示例如图2.5所示。

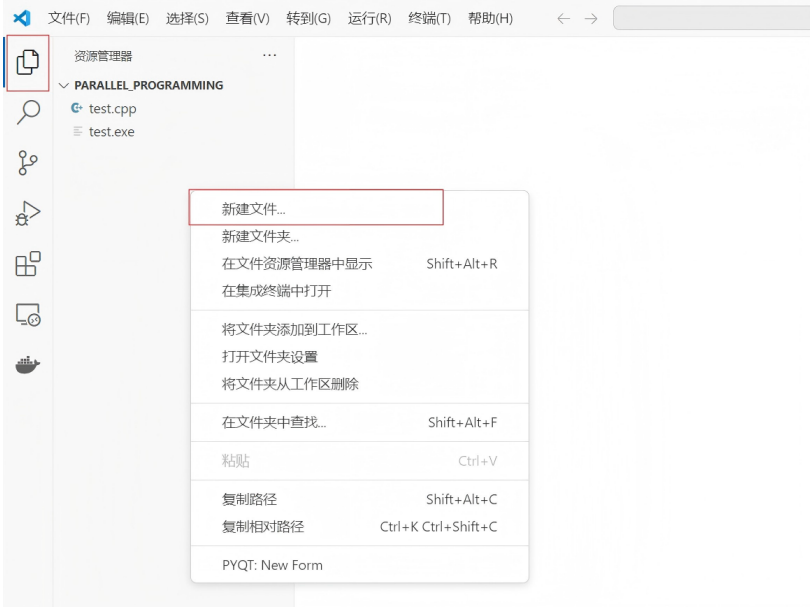

图 2.5: VSCode 创建文件示例

- **编译执行**

TDM-GCC 下载好之后会自动加入系统路径，点击 VSCode 顶部栏的终端选项，新建终端，在新建好的终端中输入 gcc –version 即可在终端打印出 GCC 编译器版本。然后在终端中输入编译命令，编译完成之后运行可执行文件即可。编译执行程序的过程如图2.6所示。

图 2.6: VSCode 编译执行程序示例

2.2 Linux 系统和 ARM 平台

ARM 架构是进阶精简指令机器 (Advanced RISC Machine) 的简称，最早是一个 32 位精简指令集（RISC）处理器架构，目前已发展到 64 位。ARM 由于能耗方面的优势，已广泛地应用于移动终端

等嵌入式场景。在保持能耗优势的前提下，ARM 处理器的性能也在不断提升，近年来逐渐进入数据中心领域，取代一部分 x86 平台的应用场景。因此，本学期建议同学们在实验中练习 ARM 平台的并行编程，特别是 x86 平台并行效果与 ARM 平台并行效果的对比。

2.2.1 WSL

WSL 是 Windows Subsystem for Linux 的简写，是 Windows 较新版本自带的一个组件，实现了在 Windows 上直接进行 Linux 开发，避免了安装运行虚拟机的复杂性。希望方便地在 Linux 系统中进行 SIMD 和多线程编程实验，即可采用这种方式，并行程序的性能实验结果与在裸机上运行的 Linux 系统中的性能表现差别不大。

WSL 对于系统有一定需求，你可以在这里找到相应说明。你可以通过升级操作系统以满足需求；但若你的机器是 32 位的，则无法使用 WSL。WSL 的安装说明可以在这里找到。你可能会依次遇到这个错误和这个错误。你可能需要升级 WSL kernel，并借助官方工具将 Windows 升级至最新版本。下面的一些软件安装、使用说明都是以在 WSL 上安装了 Unbuntu 20.04 Linux 发布版本为例。

- **WSL 下 SIMD 和多线程程序编译**

```
# 系统更新（定期做即可）和GCC编译器安装
sudo apt-get update                    # 获取Unbuntu系统和软件更新信息
sudo apt-get upgrade                   # 进行系统和软件更新
sudo apt-get install build-essential
sudo apt-get install gcc
sudo apt-get install g++

# 编译SSE/AVX程序，native可以改为本机架构名
g++ -O2 -march=native -o simd_mm simd_mm.cpp
# 编译Pthread程序，新版本GCC中编译选项已改为-pthread
g++ -O2 -pthread -o hello hello.cpp
# 编译OpenMP程序
g++ -O2 -fopenmp -o hello hello.cpp
```

在 Linux 系统下我们可以使用 QEMU 模拟多种架构，这是一款开源的硬件模拟器。QEMU 支持多种模式，我们的课程中主要用到的是 User mode 和 System mode。在 User mode 即用户模式下，QEMU 可运行不同架构下的可执行文件，功能类似于一个中间层，接受程序的指令并翻译为当前系统能够识别的指令、系统调用等。而在 System mode 即系统模式下，QEMU 会模拟一个完整的计算机系统，包括外围设备等。QEMU 模拟器对 ARM 架构有着良好的支持，借助 QEMU 就可以很好的调试和研究 ARM 环境。在 QEMU 模拟器上实现 ARM 环境主要需要进行如下步骤。

- **安装 QEMU 模拟器**

```
sudo apt-get install qemu
sudo apt-get install qemu-system
sudo apt-get install qemu-user
sudo apt install qemu-user-static
# whatever you need
# 有可能缺少依赖和库文件，根据提示安装即可
```

- **安装 ARM 的交叉编译工具链**

```
# 实际上我们可以看到有gcc-arm-linux-gnueabi、gcc-arm-linux-gnueabihf和gcc-aarch64-linux-gnu三种格式
# 具体的差异同样推荐同学们自行查阅了解，其中第一种不支持模拟SIMD
# 推荐同学们安装后两种交叉编译链
sudo apt-get install gcc-arm-linux-gnueabihf
sudo apt-get install g++-arm-linux-gnueabihf
sudo apt install gcc-aarch64-linux-gnu
sudo apt install g++-aarch64-linux-gnu
```

- **WSL 模拟 ARM 平台 SIMD 和多线程程序编译**

```
# 编译Neon程序，cortex-a76可替换成实际架构名
# 如模拟aarch64架构，不用-mfpu选项。模拟方式只能验证程序正确性，不能反映ARM SIMD真实性能
arm-linux-gnueabihf-g++ -O2 -o neon_mm -mcpu=cortex-a76 -mfpu=neon-vfpv4 neon_mm.cpp
# 如使用vdivq_f32相关指令，由于其要求系统架构为A64，而通过arm-linux-gnueabihf-g++进行编译目标为32位，因此会产生报错，此时换用：
```

```
aarch64-linux-gnu-g++ -static -o test -march=armv8.2-a NEON.cpp
# 编译Pthread程序
arm-linux-gnueabi-g++ -O2 -pthread -o hello hello.cpp
# 编译OpenMP程序
arm-linux-gnueabi-g++ -O2 -fopenmp -o hello hello.cpp
# 运行程序
qemu-aarch64 neon_mm     # 模拟64位ARM架构下执行程序
qemu-arm hello           # 模拟32位ARM架构下执行程序
hello                    # 隐含调用了qemu-arm模拟器
```

2.2.2 Linux 虚拟机

WSL 本质上也是 Linux 虚拟机，同学们也可以利用 VMware 等虚拟机软件配置一个真正的 Linux 虚拟机。

Ubuntu 是一种比较受欢迎的 Linux 发行版，它是由 Canonical Ltd. 公司及其社区开发和维护的。Ubuntu 以易用性、稳定性和安全性而闻名，被广泛用于个人计算机、服务器和云平台。它的镜像文件可在清华大学开源软件镜像站等地方找到。

关于如何使用 VMware 初始化配置一台 Ubuntu，网上有很充足的说明，同学们可以自行了解学习。在打开 Ubuntu 后，其软件包的安装和上述 WSL 一致。

2.2.3 华为鲲鹏服务器

华为鲲鹏计算产业是基于鲲鹏处理器构建的全栈 IT 基础设施、行业应用及服务。鲲鹏通用计算平台适配各行业多样性计算、绿色计算需求，致力于打造最强算力平台。鲲鹏 HPC（High Performance Computing）提供了超高浮点计算能力解决方案，可用于解决计算密集型、海量数据处理等业务的计算需求，如科学研究、气象预报、计算模拟、军事研究、CAD/CAE、生物制药、基因测序、图像处理等，缩短需要的大量计算时间，提高计算精度。如希望在实际 ARM 平台上评测真实 ARM 并行性能，可采用鲲鹏平台。

在鲲鹏服务器上，SIMD 和多线程程序的编译运行与 WSL+QEMU 环境下相似。

```
# 编译Neon程序
# 在真正ARM机器上运行，实验结果是ARM SIMD真实性能
gcc/g++ -O2 -o neon_mm -mcpu=cortex-a76 -mfpu=neon-vfpv4 neon_mm.cpp
# 编译Pthread程序
gcc/g++ -O2 -pthread -o hello hello.cpp
# 编译OpenMP程序
gcc/g++ -O2 -fopenmp -o hello hello.cpp
```

华为鲲鹏服务器可采用 putty/psftp、MobaXterm、Xshell（Windows 推荐使用，官网下载免费的学生版就够用）、Royal TSX（MacOS 推荐使用）等工具以 ssh 方式远程登录服务器、传输程序和数据等。**服务器的 ip 为 10.137.144.91，端口号为 9001**。每位同学的账户名为 **s+ 学号，例如，s2114514**，初始密码与账户名相同，同学们登录账户后可自行修改密码。

登录服务器后默认节点为 master 节点，需要使用 ssh 连接进 openeuler 虚拟机集群中，账号密码同连接 master 节点，ssh 指令如下：

```
# 只有在集群内编译运行，才能保证性能
# 将 sTest 换成自己的账号
ssh sTest@master_ubss1
```

具体测试方法见 2.2。

基于鲲鹏服务器搭建的集群和基于金山云搭建的虚拟集群都是公共实验环境，通过作业管理系统实现同学们的实验程序编译、提交、资源分配和运行。因此，登录节点只能进行简单的编译、任务提交等工作，禁止在登录节点直接执行程序。同学们应通过作业管理系统将自己的实验程序提交，等待作业管理系统分配资源、调度执行完毕后再获取结果，详见下一节 MPI 实验环境的介绍。建议同学们先

在本地台式机、笔记本（x86 平台和模拟 arm 平台）进行程序的编写和正确性调试，进行性能评测时再登录两个平台提交任务。

2.2.4 VSCode 连接远程服务器

上一小节提到了 Xshell 等远程连接服务器的工具，在具体开发过程中，上述工具可能需要通过 vim 等指令编写代码，较为复杂。为了方便使用，这里推荐使用 VSCode 连接远程服务器，具体操作可以在这里找到。连接成功后，同学们可以通过 VSCode 的控制台指令来编译运行、通过 VSCode 来编写代码。

3 MPI 实验环境

在本学期中期，同学们要进行 MPI 编程实验。MPI 是一种由学术界工业界联合设计的标准化可移植消息传递标准，可在多种并行计算体系结构上运行。MPI 提供了在并行计算机中进行节点通信的协议，支持点对点通信和组通信，其目标是高性能、可伸缩性和可移植性。MPI 直到现在仍是高性能计算中使用的主导编程模型。

3.1 自行搭建

MPI 适用于各种并行计算架构，你可以自行搭建不同系统下（甚至无需集群环境，单机系统亦可部署，但性能测试结果不反应集群环境中并行计算实际性能）的 MPI 实验环境。

当然，同 SIMD 和多线程实验，最终测试要求同学们必须使用华为鲲鹏服务器，以下环境配置仅供参考。

3.1.1 MS-MPI

MS-MPI 是 MPI 标准的 Microsoft 实现，用于在 Windows 平台上开发和运行 MPI 程序。它在 Windows 系统上具有着高性能，同时很容易移植使用 MPICH 的现有代码。

你可以在这里下载最新 MS-MPI 代码，并在这里找到安装说明和运行简单测试程序的方法。

3.1.2 OpenMPI

OpenMPI 是一个广泛使用的 MPI 实现，在 Linux 系统或是 WSL 中，你可以直接自行部署 OpenMPI 环境。这里提供了最新版本的 OpenMPI 安装包，参考部署指南进行部署和验证。

3.1.3 MPICH

MPICH 是 MPI 标准的一个重要实现。Windows 版本的 MPICH 由于某些原因不再更新，所以 Win7 及以后的版本都很难成功安装 MPICH，如果你需要使用 MPICH 进行实验，那么强烈推荐你在 Linux 环境或是 WSL 环境下进行部署配置。

你可以在这里获取到适用于各种系统的最新版本。你可以根据这里的安装指南进行安装和使用。

3.2 鲲鹏服务器

3.2.1 pbs 脚本配置

提交任务前需配置脚本，例如运行 4 个进程 mpi 程序可以编写 mpi.sh 脚本内容如下：

```
#mpi.sh
#!/bin/sh            # 这是脚本的解释器行，表示使用Bash shell解释脚本
#PBS -N mpi          # 任务程序为mpi
#PBS -l nodes=4      # 需要四个节点来计算

/usr/local/bin/pssh -h $PBS_NODEFILE mkdir -p /home/sTest 1>&2                    # 在分配到的4个计算节点上创建对应的路径
scp master_ubss1:/home/sTest/mpi /home/sTest                                  # mpi为编译之后的可执行程序
/usr/local/bin/pscp -h $PBS_NODEFILE /home/sTest/mpi /home/sTest 1>&2             # 获取master节点中的mpi可执行程序，并分发到每个计算节点
mpiexec -np 4 -machinefile $PBS_NODEFILE /home/sTest/mpi              # 在4个计算节点上运行可执行程序mpi
```

注意，对单节点非 MPI 实验，每个任务只能使用 1 个节点（核心）、运行一个进程，节点数设置为 1，无须使用 mpiexec，直接执行程序即可。对于非 MPI 实验，可以编写 test.sh 脚本如下：

```
#test.sh
#!/bin/sh
#PBS -N test         # 任务程序为test
#PBS -l nodes=1      # 需要一个节点来计算

/usr/local/bin/pssh -h $PBS_NODEFILE mkdir -p /home/sTest 1>&2
scp master_ubss1:/home/sTest/test /home/sTest
/usr/local/bin/pscp -h $PBS_NODEFILE /home/sTest/test /home/sTest 1>&2
/home/sTest/test
```

注意，直接粘贴脚本到服务器可能会出现乱码，如提示乱码在服务器内重新手动编辑即可。

3.2.2 作业提交

我们利用鲲鹏服务器搭建虚拟机集群供同学们进行 MPI 编程实验，包括一个 master 节点 (即虚拟机的 master_ubss1) 和 22 个计算节点，每个节点 8 核，12GB 内存。登录管理节点 IP 地址 10.137.144.91（需要链接校园网），端口号为 9001，登录用户名和密码均为 s+ 学号，例学号为 2111111，则用户名和密码均为 s2111111。

登陆管理节点后**（请不要在 master 节点直接运行任何程序!）**，先使用 mpicc 或 mpic++ 将程序编译成可执行文件（与自行搭建的 OpenMPI/MPICH 系统类似；非 MPI 的单节点实验使用 gcc 或毕升），同时配置好 pbs 脚本（参见上一小节），然后在可执行文件所在目录下输入命令：

```
qsub test.sh
```

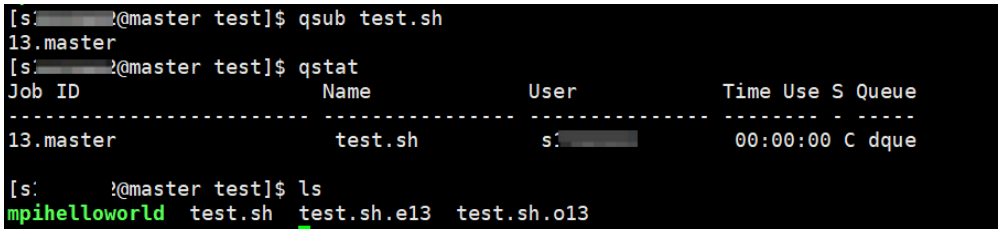

图 3.7: 鲲鹏服务器作业提交

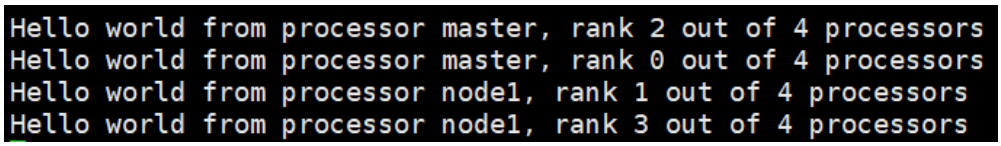

图 3.8: test.sh.o13

提交后会得到一个作业编号，如图3.7, 比如 13.master。输入 qstat 可以看到目前队列中作业的状态：

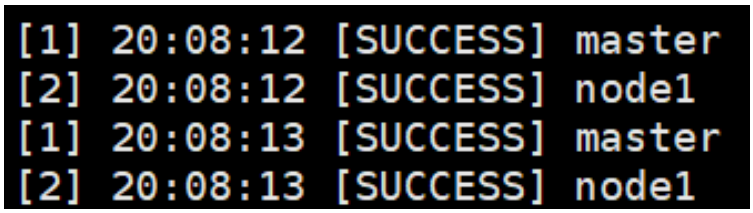

图 3.9: test.sh.e13

```
Q–正在排队 R–正在运行 C–运行完毕
```

执行完成后，在当前目录下会生成两个文件，一个是标准输出，文件名为 test.sh.o13 （13 是作业编号），如图3.8；另一个是错误输出，包括提示或报错，文件名为 test.sh.e13，如图3.9。

4 GPU 实验环境

4.1 英伟达 CUDA

CUDA（Compute Unified Device Architecture）是显卡厂商 NVIDIA 推出的运算平台，是一种通用并行计算架构，该架构使 GPU 能够解决复杂的计算问题。它包含了 CUDA 指令集架构以及 GPU 内部的并行计算引擎。开发人员现在可以使用 C 语言来为 CUDA 架构编写程序。下面介绍在个人电脑上如何进行环境搭建。

4.1.1 个人电脑环境搭建

在个人电脑上安装 CUDA 首先要满足的硬件条件是：电脑必须要有 NVIDIA 独立显卡。满足此条件之后，按照此方法可以查询电脑 CUDA 支持的版本，然后去官网下载工具包，并按照官网文档中步骤进行安装。官网链接：`https://developer.nvidia.com/zh-cn/cuda-toolkit`。

4.2 AMD ROCm

4.2.1 AMD 云端学习平台

本学期的 GPU 编程实验基于 AUP Learning Cloud 平台开展，AUP Learning Cloud 是由 AMD 开发的一个定制化 JupyterHub 平台，此平台集成了完整的 AMD 加速工具链，旨在为并行计算、人工智能学习提供一站式实验环境。

同学们可以通过在浏览器中访问`https://www.openhw.io/`打开 AUP Learing Cloud 登录页面，如图4.10所示，助教会为大家分配账号，根据助教分配的本地账号密码进行登录，每个账号密码由五位同学共享，请大家注意**不要修改密码，账号和密码仅限本课程实验内容使用**。

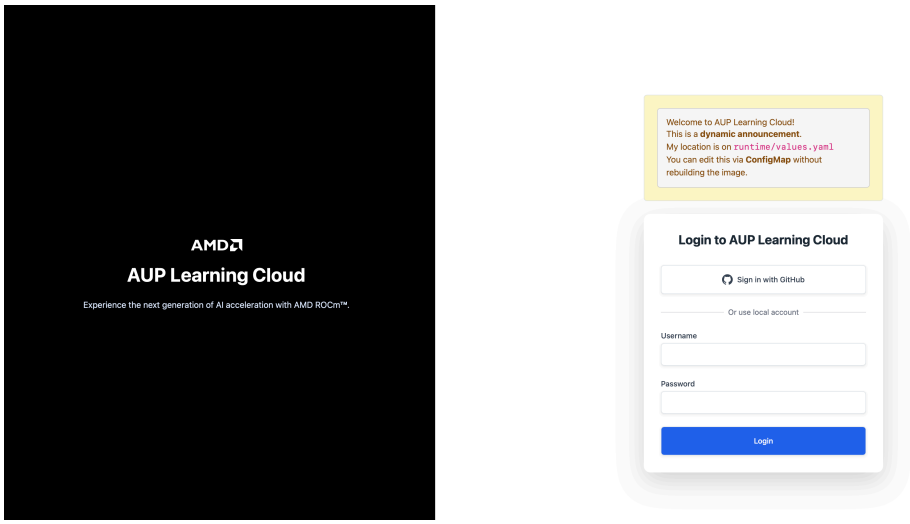

图 4.10: AMD 云端学习平台登录界面

登录后会进入课程选择界面，如图4.11所示，选择 HIP Programming Course，在页面底部设置镜像运行时间，请大家设置合理的运行时间，计时结束时会关闭链接，**未放置在用户存储目录/home/jovyan 的内容会被重置**，因此提醒大家在镜像结束之前将工作内容迁移至/home/jovyan 目录。计时结束之后如果有需要还可以重启镜像，为避免资源浪费，镜像时间不需要设置太长，设置完成后点击 Launch。

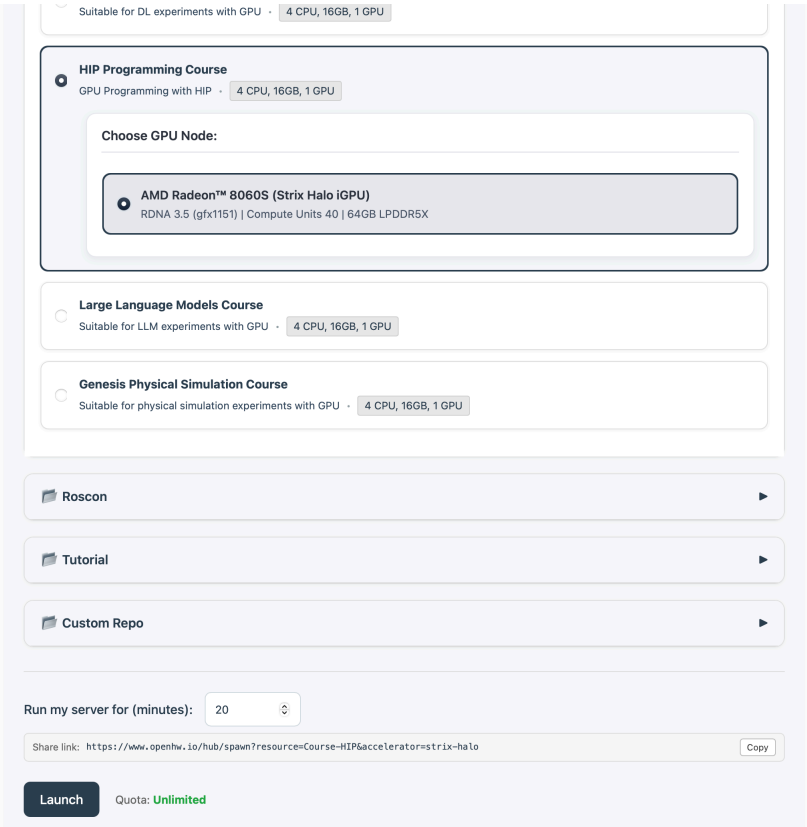

图 4.11: 课程选择和镜像时间设置

关于 AMD GPU 编程范式及 API 的学习，可以在左侧边栏点击 HIP 目录，然后选择具体案例 Lab，例如选择 Lab1_Vector_Addition，进入该目录后打开 ipynb 文件进行学习，如图4.12所示。

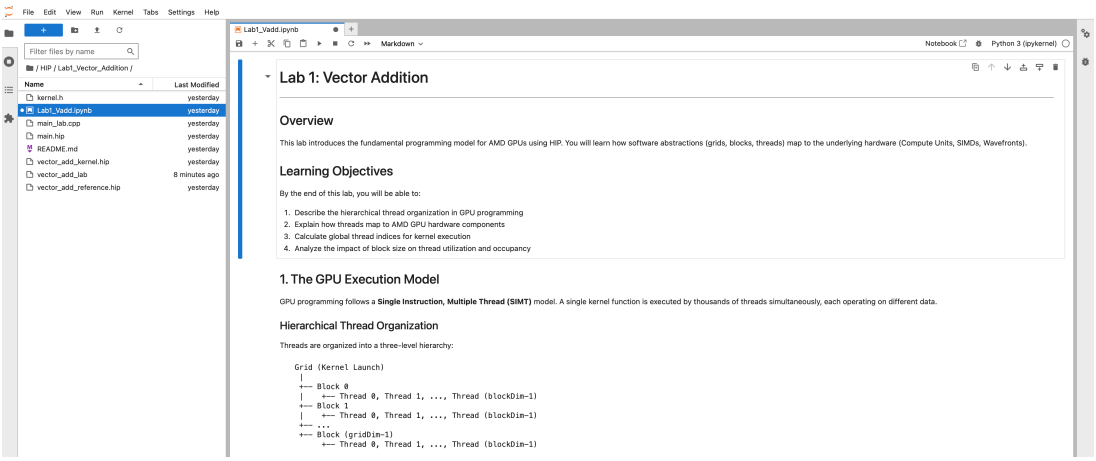

图 4.12: 案例学习

请大家根据自己的选题在此平台上进行编程实验，使用完成后，请正确退出，首先保存你的数据，存放至/home/jovyan 目录下，建议在此目录下新建一个自己的目录，和其他同学的区分开，然后点击左上角 File，点击 Hub Control Panel 停止镜像，如图所示4.13，最后退出登录即可，注意不要长时间占用服务器资源。

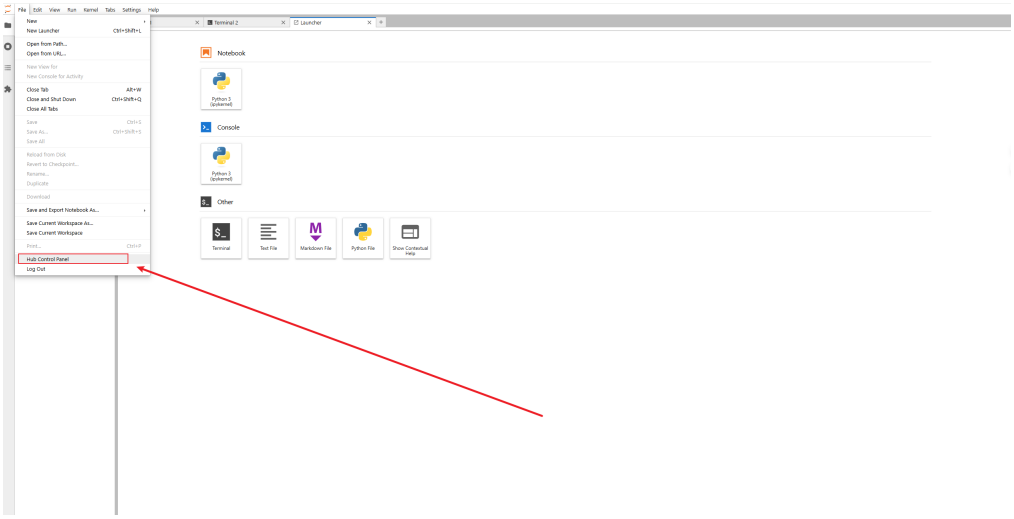

图 4.13: 停止镜像

5 性能测试方法

测试程序的性能，我们需要得到程序实际需要的空间和时间，其中编译所需空间时间可以不考虑。计时的机制有：

- 跨平台：clock() / CLOCKS_PER_SEC，精度不高
- Linux：gettimeofday()
- Windows：QueryPerformance()

测试方法：(1) 给定问题规模 n; (2) 按需求设计测试数据。

5.1 问题规模

问题规模的确定因素为执行时间和执行次数，即对于一个程序，我们究竟需要测试多少次，每次测试多少数据。以插入排序为例子，其中最坏的情况为 $\Theta(n^2)$，平方函数至少需要 3 个点，并且在测试时需要 3 个以上的 n，n 需要渐进的取到，同时当 n 取值较小时，同一个 n 需要测试多次来取到精确的结果。一种可能的规模设计为：对于 $n = 0 - 100$，精细测试 0, 10, 20，..., 100；对 n>100，稀疏测试 200, 300, 400，...。

5.2 测试数据

测试数据的设计需要仔细思考，如果测试数据过于特别，将无法得到程序真正的性能。以输入排序为例，最坏以及最好的情况下测试数据分别为递减序列和递增序列。但是如果我们需要测试程序的平均情况，这时候需要随机产生大量的测试数据。

5.3 样例

以插入排序为例子，如果我们需要测试插入排序的最坏情况，测试数据就需要是递减序列。

```
void main(void) {
   int a[1000], step = 10;
   clock_t start, finish;
   for (int n = 0; n <= 1000; n += step) {
      // get time for size n
      for (int i = 0; i < n; i++)
         a[i] = n - i; // initialize
      start = clock();
      InsertionSort(a, n);
      finish = clock();
      cout << n << ' '
      << (finish - start)/float(CLOCKS_PER_SEC) << endl;
      if (n == 100) step = 100;
    }
}
```

测试规模我们取 $n = 10, 20, \ldots, 100, 200, \ldots, 1000$，测试结果如表1所示。

表 1: 测试结果

n	时间 (s)	n	时间 (s)
0	0	100	0.06
10	0	200	0
20	0	300	0
30	0	400	0
40	0	500	0
50	0	600	0.05
60	0	700	0.06
70	0	800	0.05
80	0	900	0.06
90	0	1000	0.11

同时我们需要重复测试，从而提高精度。程序可以改为：

```
void main(void)
{
   int a[1000], n, i, step = 10;
   long counter;
   float seconds;
   clock_t start, finish;
   for (n = 0; n <= 1000; n += step) {
      // get time for size n
      start = clock(); counter = 0;
      while (clock() - start < 10) {
         counter++;
         for (i = 0; i < n; i++)
            a[i] = n - i; // initialize
         InsertionSort(a, n);
         }
      finish = clock();
      seconds = (finish - start)/float(CLOCKS_PER_SEC);
      cout << n << ' ' << counter << ' ' << seconds
           << ' ' << seconds / counter << endl;
      if (n == 100) step = 100;}
}
```

这样得到的测试结果中，当 n 取值较小时，重复多次来获得更加精确的结果，测试结果如表2所示。

表 2: 测试结果

n	重复次数	总时间 (s)	每次排序时间 (s)
0	65	0.06	0.000923077
10	294	0.05	0.000170068
20	348	0.06	0.000172414
30	145	0.05	0.000344828
40	120	0.06	0.0005
50	179	0.05	0.00027933
60	141	0.06	0.000425532
70	70	0.05	0.000714286
80	99	0.06	0.000606061
90	60	0.05	0.000833333
100	60	0.06	0.001
200	12	0.05	0.00416667
300	9	0.06	0.00666667
400	4	0.05	0.0125
500	3	0.06	0.02
600	2	0.05	0.025
700	1	0.06	0.06
800	1	0.05	0.05
900	1	0.11	0.11

5.4 Windows 下精确计时

windows 下精确计时可使用 QueryPerformance 系列函数进行计时（需包含 windows.h 头文件），具体如下：

```
#include <iostream>
#include <windows.h>
#include <stdlib.h>

using namespace std;

const int N = 10240;            // matrix size

Double b[N][N], col_sum[N];

Void init(int n)        // generate a N*N matrix
{
  for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
      b[i][j] = i + j;
}
int main()
{
    long long head, tail, freq;        // timers
    init(N);
    // similar to CLOCKS_PER_SEC
    QueryPerformanceFrequency((LARGE_INTEGER *)&freq);
    // start time
    QueryPerformanceCounter((LARGE_INTEGER *)&head);
    for (int i = 0; i < 1000; i++) {
        col_sum[i] = 0.0;
        for (int j = 0; j < 1000; j++)
        col_sum[i] += b[j][i];
     }
    // end time
    QueryPerformanceCounter((LARGE_INTEGER *)&tail);
    cout << "Col: " << (tail - head) * 1000.0 / freq
   << "ms" << endl;
}
```